

Module 2

(1.) Fraud transaction detection (Linear search)

Problem Statement

A bank stores transaction IDs in the order they occur. Since transactions are not sorted, check whether a suspicious transaction ID exists.

★ Logic used:-

- ⇒ Input n transactions IDs in array format.
- ⇒ Input target transaction ID inside the array
- ⇒ Traverse array from first to last element
- ⇒ From traversing target is found, so print "fraud transaction found". otherwise print "Transaction not found".

★ code:-

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "enter the size of array; ";
    cin >> n;
    int arr[n];
    cout << "enter the element of an array!";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    int target;
    cout << "enter the target value!";
    cin >> target;
    // all found = false
```

```

16     cin >> target;
17
18     bool found = false;
19
20     for(int i=0;i<n;i++) {
21         if(arr[i] == target) {
22             found = true;
23             break;
24         }
25     }
26
27     if(found)
28         cout << "Fraud Transaction Found";
29     else
30         cout << "Transaction Not Found";
31
32     return 0;
33 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps1.cpp

PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe

Enter the number of transactions: 5

Enter the transaction amounts: 1201

1305

1450

140

1502

Enter the target transaction amount: 1450

Fraud Transaction Found

```

for (int i = 0; i < n; i++) {
    if (arr[i] == target) {
        found = true;
        break;
    }
}

```

```

if (found) {
    cout << "found transaction found";
} else {
    cout << "Transaction not found";
}
return 0;
}

```

★ Time complexity:- $O(n)$

(2.) Server log search (Binary search)

★ Problem statement:-

A cloud company stores server log IDs in sorted order. Find whether a target logic ID exists

★ Logic used:-

- 1) Input n server log ID in array format with sorted order.
- 2) Input target server log ID in inside the array
- 3) Initialize low = 0 & high = $n-1$
- 4) Find middle element.
- 5) If middle element equal to target, then stop.
- 6) if target is smaller, search in left half.
otherwise search in right half.
- 7) Repeat until target not found or low > high.

code:-

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "enter the size of array: ";
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    cout << "enter the element of array: ";
```

```
    for (int i = 0; i < n; i++)
```

```
        cin >> arr[i];
```

```
    int target;
```

```
    cout << "enter the value of target: ";
```

```
    cin >> target;
```

```
    int st = 0, end = n - 1;
```

```
    bool found = false;
```

```
    while (st <= end) {
```

```
        int mid = st + (end - st) / 2;
```

```
        if (arr[mid] == target) {
```

```
            found = true;
```

```
            break;
```

```
        } else if (target < arr[mid]) {
```

```
            high = mid - 1;
```

```
        } else {
```

```
            low = mid + 1;
```

```
        }
```

```
    } if (found) {
```

```
        cout << "Log found";
```

```
    } else {
```

```
        }
```

```
        cout << "Log not found";
```

```
        return 0;
```

```
23
24         if(arr[mid] == target) {
25             found = true;
26             break;
27         }
28         else if(target < arr[mid])
29             high = mid - 1;
30         else
31             low = mid + 1;
32     }
33
34     if(found)
35         cout << "Log Found";
36     else
37         cout << "Log Not Found";
38
39     return 0;
40 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps2.cpp

PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe

Enter the number of logs: 7

Enter the log elements: 1001

1005

1010

1015

1020

1030

1041

Enter the target element: 1015

Log Found

(3.1) Sort employee salaries (Bubble sort)

* Problem Statement:-

Sort employee salaries in ascending order.

* Logic used:-

- ⇒ compare adjacent elements.
- ⇒ swap if left element is greater.
- ⇒ Repeat for all passes.
- ⇒ Largest element reaches the end after every pass.

* code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            swap(arr[i], arr[i + 1]);
        }
    }
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

```
22     cout << "Sorted elements. ";  
23     for(int i=0;i<n;i++)  
24         cout << arr[i] << " ";  
25  
26     return 0;  
27 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps3.cpp

PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe

Enter the number of elements: 5

Enter the elements: 4500 3200 5500 2800 6000

Sorted elements: 2800 3200 4500 5500 6000

(4.) sort website response Time (Insertion sort)

* Problem Statement:-

sort website response time in ascending order.

* Logic used:-

- ⇒ Assume first element is sorted.
- ⇒ Pick next element as key.
- ⇒ Shift greater ~~element~~ elements to the right.
- ⇒ Insert key at correct position
- ⇒ Repeat until array become sorted.

* code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 & arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

```
}

cout << "Sorted elements: ";
for(int i=0;i<n;i++)
    cout << arr[i] << " ";

return 0;
```

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Users\manoj\OneDrive\Documents\DSA C++> g++ ps4.cpp

Users\manoj\OneDrive\Documents\DSA C++> .\a.exe

Enter the number of elements: 5

Enter the elements: 250 120 320 170 100

(5.) Prioritize Bug reports (selection sort)

6

* Problem Statement:-

sort bug priority scores in ascending order

* Approach^{Logical}

- Find minimum element.
- swap it with current position.
- Repeat for remaining array
- continue until array become sorted

* code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        swap(arr[i], arr[minIndex]);
    }
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

```
23     swap(arr[i], arr[minIndex]);
24 }
25 cout << "Sorted elements: ";
26 for(int i=0;i<n;i++)
27     cout << arr[i] << " ";
28
29 return 0;
30 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps5.cpp

PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe

Enter the number of elements: 5

Enter the elements: 9 2 7 1 5